and whether deduplication could effectively reduce storage or improve compression efficiency.

As illustrated in Fig. 2, the results reveal a low level of redundancy across all datasets. In the clinical dataset, for example, the top 10 tokens account for 27.53% of the text, but bigrams, trigrams, and four-grams contribute significantly less, with bigrams at 5.03%, trigrams at 1.86%, and four-grams at just 0.93%. A similar pattern is observed in the code and math datasets, where repeated n-grams take up only a small portion of the text.

These findings suggest that deduplication techniques, which have good performance on machine-generated data, have limited potential for LLM-generated synthetic datasets. Unlike conventional machine-generated data, which often contains repetitive phrases and structural redundancies, LLM-generated text is more diverse, reducing the effectiveness of deduplication-based compression methods. One possible scenario in the future is that when the same LLM generates text from different sources, there may be opportunities for file-level deduplication. However, this falls beyond the scope of the research topic discussed in this paper.

3.2 Information Evaluation of the machine-generated data, human-generated data, and LLM-generated data.

Lossless compression aims to encode a sequence of symbols $x_{1:n}$ into a bitstream with minimal expected length while ensuring perfect reconstruction of the original data. According to Shannon's source coding theorem [45], the optimal compression efficiency is bounded by the entropy of the source distribution. Given a source probability distribution $\rho(x)$, the Shannon entropy is defined as:

$$H(\rho) := \mathbb{E}_{x\sim\rho}[-\log_2 \rho(x)] \tag{1}$$

The Shannon entropy represents the theoretical lower bound on the average number of bits needed to encode a symbol drawn from ρ. A well-designed compression scheme assigns shorter codes to more frequent symbols and longer codes to rare symbols, achieving an average code length L satisfying:

$$L \geq H(\rho) \tag{2}$$

where equality holds under an optimal entropy coder such as Huffman coding or Arithmetic coding in the limit of large sequences. This entropy bound establishes the fundamental limits of lossless data compression, beyond which further compression requires exploiting higher-order dependencies or introducing loss.

To evaluate the impact of tokenization granularity on information representation, we compute the entropy per byte for character-level, subword (BPE), and word-level tokenization. The entropy per byte is computed as follows:

$$H_{\text{byte}} = \frac{H_{\text{token}}}{L_{\text{avg}}}$$

whereH_{token} is the Shannon entropy per token, calculated as:

$$H_{\text{token}} = -\sum_{t\in T} P(t)\log_2 P(t)$$

where T is the set of tokens, and $P(t)$ is the probability of token t. L_{avg} is the average token length in bytes, defined as:

$$L_{\text{avg}} = \frac{\sum_{t\in T} P(t)\cdot L(t)}{\sum_{t\in T} P(t)}$$

where $L(t)$ is the byte length of token t. This formula applies to character-level, subword (BPE), and word-level tokenization by adjusting the token set T accordingly.

We also use *Mutual Information Between Consecutive Words*, which measures how much knowing one word helps predict the next word. It is computed as:

$$MI(W_i, W_{i+1}) = \sum_{w_i\in W}\sum_{w_{i+1}\in W} P(w_i, w_{i+1})\log_2 \frac{P(w_i, w_{i+1})}{P(w_i)P(w_{i+1})}$$

where W_i and W_{i+1} are consecutive words in the dataset, $P(w_i, w_{i+1})$ is the joint probability of two consecutive words w_i and w_{i+1}, and $P(w_i)$ and $P(w_{i+1})$ are the marginal probabilities of words w_i and w_{i+1}, respectively. This metric provides insights into the structural dependencies within different tokenization schemes, helping assess how well each method retains contextual information.

Table 2 presents a comparative analysis of these characteristics, where Char-E, BP-E, and W-E indicate Char-Entropy, BPE-Entropy and Word-Entropy. To ensure a fair comparison, the human-generated dataset and the LLM-generated dataset both originate from the same source (Wikipedia), with each sentence conveying the same meaning. The only difference is that the LLM-generated data has been rewritten by an LLM. For the machine-generated data, i.e., TPC-H, we collected the comments field from its tables to ensure a fair comparison with the other two datasets.

Table 2: Redundancy and token frequency characteristics of LLM-generated text vs. structured datasets.

Dataset	Char-E	BP-E	W-E	Mutual Info
LLM-Generated	4.67	2.51	2.57	2.95
Human-Generated	4.63	2.53	2.59	2.73
TPC-H	4.34	1.72	0.94	1.23

We observe that the entropy measures of both LLM-generated data and human-generated data are similar across character, BPE, and word levels. Moreover, these entropy values are relatively low, indicating that traditional compression methods, including state-of-the-art approaches like Zstd, are likely to achieve only relatively limited compression ratios.

Interestingly, when examining mutual information (Mutual Info), LLM-generated data exhibits relatively higher token predictability, despite each sentence conveying the same meaning as its human-generated counterpart. As expected, TPC-H shows very low mutual information since it is randomly generated and lacks a continuous semantic structure.

3.3 Limitations of Existing Compression Methods

Built upon the aforementioned analysis, we selected three types of LLM-generated synthetic text for a preliminary examination of traditional compression methods, including a mixed-domain

Wikipedia dataset and two specific-domain datasets: math and code. For compression baselines, we used three state-of-the-art compressors—LZMA [58], Gzip [15], and Zstd [10], along with three Neural Network-based (NN-based) compressors—NNCP [3], TRACE [34], and PAC [35] to conduct a comprehensive study on the compressibility of LLM-generated text.

Table 3: Compression ratios of different datasets using Gzip, LZMA, Zstd-22, NNCP, TRACE and PAC.

Dataset	Gzip	LZMA	Zstd	NNCP	TRACE	PAC
Wiki	2.73	4.24	4.17	5.04	4.11	4.17
Code	3.47	4.90	4.85	6.30	4.53	5.08
Math	4.30	5.99	5.90	7.76	5.34	6.39

The compression results are given in Table 3. We can find that, while modern compressors such as LZMA and neural-based methods like NNCP and PAC outperform traditional dictionary-based approaches such as Gzip and Zstd-22, the overall compression ratios remain relatively limited. Among the traditional methods, LZMA achieves the highest compression efficiency, with ratios ranging from 4.24 (Wiki) to 5.99 (Math). Zstd-22 performs similarly to LZMA but offers slightly lower compression ratios across all datasets. Neural-based methods demonstrate further improvements, with NNCP consistently achieving the best performance among the listed approaches, obtaining 6.30 (Code), 7.76 (Math), and 5.04 (Wiki). TRACE and PAC provide competitive results but still fall within a similar compression range.

Despite these incremental improvements, all existing methods struggle to achieve compression ratios beyond a single-digit range. The results highlight the inherent limitations of conventional entropy and dictionary-based compressors, as well as the current generation of neural compressors, when applied to unstructured and irregular text datasets. It calls for developing efficient compression techniques tailored to the unique characteristics of the LLM-generated text.

4 LLM-based Compression Framework

To resolve the irregularity of the LLM-generated text, we propose to leverage LLMs to compress the LLM-generated text. In this section, we present a detailed LLM-based framework that achieves remarkable lossless compression performance compared to traditional compression approaches.

4.1 Conceptual Framework

In this section, we provide a conceptual description of our LLM-based compression framework. It consists of two steps: a) context-based prediction and b) encoding coding.

4.1.1 Context-based prediction. Context-based prediction is a foundational technique in neural lossless compression[3, 14, 34], aiming to estimate the probability distribution of a symbol x_t conditioned on its preceding context $x_{<t}$. Accurate probability estimation affords efficient entropy coding and reduces the redundancy in data representation.

Given a sequence of symbols $X = (x_1, x_2, \ldots, x_T)$ in length T, the probability distribution of the entire sequence can be factorized using the chain rule of probability:

$$P(X) = \prod_{t=1}^{T} P(x_t \mid x_{<t}), \tag{3}$$

where $x_{<t} = (x_1, x_2, \ldots, x_{t-1})$ represents the preceding symbols. The objective of context-based prediction is to accurately model the conditional probability $P(x_t \mid x_{<t})$ to minimize the entropy of the encoded sequence.

4.1.2 Entropy coding. Entropy coding relies on the fundamental principle that the optimal encoding length of a symbol x_t is given by its self-information:

$$L(x_t) = -\log_2 P(x_t \mid x_{<t}). \tag{4}$$

Thus, the expected length per symbol, also known as the entropy, is given by:

$$\begin{aligned} H(X) &= \mathbb{E}[-\log_2 P(x_t \mid x_{<t})] \\ &= -\sum_{x_t} P(x_t \mid x_{<t}) \log_2 P(x_t \mid x_{<t}). \end{aligned} \tag{5}$$

Efficient entropy coding is achieved when the estimated probability distribution $\hat{P}(x_t \mid x_{<t})$ closely matches the true probability distribution $P(x_t \mid x_{<t})$, leading to minimal redundancy in encoding. On the other hand, the coding inefficiency R can be quantified as:

$$R = H(\hat{P}) - H(P), \tag{6}$$

where $H(\hat{P})$ is the entropy computed using the estimated probability distribution. The Kullback-Leibler (KL) divergence provides a measure of the discrepancy between these distributions:

$$D_{KL}(P \parallel \hat{P}) = \sum_{x_t} P(x_t \mid x_{<t}) \log_2 \frac{P(x_t \mid x_{<t})}{\hat{P}(x_t \mid x_{<t})}. \tag{7}$$

From information theory, it follows that:

$$H(P) \leq H(\hat{P}) = H(P) + D_{KL}(P \parallel \hat{P}). \tag{8}$$

This inequality implies that a more accurate probability model reduces D_{KL}, leading to a smaller gap between the estimated entropy $H(\hat{P})$ and the true entropy $H(P)$, thereby improving compression efficiency. In the ideal case where $\hat{P} = P$, the KL divergence becomes zero, achieving optimal entropy coding.

4.2 The Next-Token Prediction Framework

LLMs generate text in an autoregressive manner. In other words, LLMs predict the next token in a sequence based on previously generated tokens. The training of LLMs leverages the next-token prediction objective, which is to model the probability distributions over sequences of tokens.

The training process of LLMs essentially resembles the context-based prediction. To capture relevant contextual information, LLMs employ an embedding function f_θ, parameterized by θ, to transform the context $x_{<t}$ into a hidden state representation h_t:

$$h_t = f_\theta(x_{<t}). \tag{9}$$

Transformer-based Models (e.g., GPT-like Architectures) utilize self-attention mechanisms to capture long-range dependencies [49]:

$$h_t = \sum_{j<t} \alpha_{t,j} (W_V x_j), \tag{10}$$

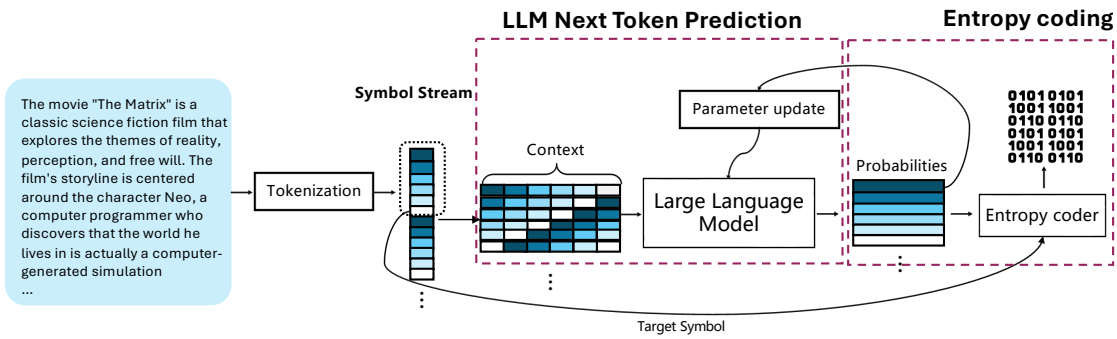

Figure 4: General process of Neural-based Compression.

where attention weights $\alpha_{t,j}$ are computed as follows:

$$\alpha_{t,j} = \frac{\exp((W_Q x_t)^\top (W_K x_j))}{\sum_{j<t} \exp((W_Q x_t)^\top (W_K x_j))}. \tag{11}$$

Once the contextual representation h_t is obtained, the probability distribution of the next token x_t is modeled using a softmax function:

$$P(x_t \mid x_{<t}) = \text{softmax}(W_o h_t), \tag{12}$$

where W_o is a learnable weight matrix that maps the hidden state to output logits. The predicted probability distribution is then utilized in entropy coding algorithms to achieve optimal lossless compression. In this work, we implement the entropy coding as the arithmetic coding [51].

Tokenization and Embedding. Prior to processing, text is tokenized into discrete units using methods such as Byte Pair Encoding (BPE) or WordPiece. Each token x_t is then mapped to a high-dimensional embedding vector e_t via a pre-trained embedding matrix:

$$e_t = \text{Embedding}(x_t). \tag{13}$$

Transformer-Based Processing. The token embeddings are processed through multiple layers of a Transformer model, which consists of self-attention and feedforward networks. The self-attention mechanism computes token dependencies using:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}} + M\right) V, \tag{14}$$

where Q, K, and V are query, key, and value matrices, respectively, and M is a causal mask preventing future tokens from influencing current token computations.

Probability Estimation and Decoding. The final hidden representation h_t of the last Transformer layer is projected onto the vocabulary space and converted into a probability distribution using a softmax function:

$$P(x_{t+1}|x_1, \ldots, x_t) = \text{softmax}(W h_t + b), \tag{15}$$

where W and b are trainable parameters.

Training Objective. The model is trained using a causal language modeling objective, or next-token prediction, minimizing the cross-entropy loss:

$$L = -\sum_{t=1}^{T} \log P(x_t|x_1, \ldots, x_{t-1}). \tag{16}$$

Gradients are computed via backpropagation, and model parameters are updated using optimization algorithms such as Adam [29].

4.3 Entropy Coding

We use arithmetic coding as the entropy coder [51]. Arithmetic coding is an advanced entropy encoding technique that efficiently represents a sequence of symbols as a single floating-point number within the unit interval $[0, 1)$. Unlike Huffman coding, which assigns fixed-length binary codes to individual symbols [24], arithmetic coding progressively refines an interval based on symbol probabilities, achieving near-optimal compression close to the Shannon entropy limit.

4.3.1 Encoding Process. Given an input sequence $X = (x_1, x_2, \ldots, x_T)$ in length T with respective probabilities $P(x_t)$, arithmetic coding proceeds as follows. Initially, the encoding interval is set to $[L, H) = [0, 1)$.

For each symbol x_t in the sequence, the current interval width is computed as:

$$R = H - L.$$

The interval is then updated based on the cumulative probability of preceding symbols:

$$L' = L + R \cdot F(x_t), \quad H' = L + R \cdot (F(x_t) + P(x_t)),$$

where $F(x_t)$ represents the cumulative probability of all symbols preceding x_t.

This process continues iteratively until all symbols have been encoded. The final representation is chosen as a value within the interval $[L_T, H_T)$, ensuring a minimal binary representation.

4.3.2 Decoding Process. Decoding follows the inverse procedure. The interval is initialized as $[L, H) = [0, 1)$, and the encoded value X is read.

At each step, the symbol x_t is identified as the one whose assigned sub-interval $[L', H')$ contains X. The decoded symbol is then